

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO



BÀI TẬP LỚN SỐ 2
PHƯƠNG PHÁP LUẬN LẬP TRÌNH
(2526II_UET.AI2013_2)

Sinh viên : Trần Đức Anh

Lớp: K70A-AI2

Mã sinh viên: 25022777

MỤC LỤC

I. Tổng quan:

1. Tổng quan thiết kế luồng thực thi trong bài.....
2. Giao diện đồ họa và kiểu tương tác.....

II. Chi tiết cài đặt:

1. Tư duy thiết kế.....
2. Giao diện đồ họa.....
3. Tuân thủ nguyên tắc và áp dụng mẫu thiết kế.....

III. Kết quả thực nghiệm:

IV. Nhận xét và tổng kết bài làm:

V. Ghi chú sử dụng AI:

VI. Nguồn tham khảo:

I. Tổng quan:

1. Tổng quan thiết kế luồng thực thi:

- Chương trình được thiết kế theo luồng tuần tự kết hợp với trừu tượng hoá giao diện thông qua interface. Toàn bộ vòng đời game do lớp Engine điều phối, hoạt động thông qua hai interface là I_Renderer và I_Interaction.
- Engine không cần biết chương trình đang chạy trên Terminal hay GUI - nó chỉ gọi các phương thức trừu tượng. Tùy vào cờ -g (gui_flag) được truyền vào khi khởi động, hệ thống sẽ truyền TerminalRenderer/TerminalInteraction hoặc SDLRenderer/SDLInteraction tương ứng.

2. Giao diện đồ họa và kiểu tương tác:

Hình thức hiển thị	Hình thức tương tác
- Vẽ hình học cơ bản (SDL_RenderDrawLine, SDL_RenderFillRect, ...) để tạo lưới bàn cờ và các ô vuông hay các đường thẳng. - Hiển thị chữ bằng SDL_ttf, có thể chọn font chữ, font size, ... - Vẽ ảnh bằng SDL_image để trang trí menu game trở nên bắt mắt hơn.	- Bàn phím (SDL_KEYDOWN) để tương tác nhập số chọn cờ bàn cờ, điều kiện thắng, chế độ chơi, ... - Chuột (SDL_MOUSEBUTTONDOWN): click để chọn chế độ chơi, chọn bot level, và vị trí đặt quân cờ. - Thực tế bài làm lần này cho phép người chơi chọn cả chế độ chơi, bot level bằng cả bàn phím cả chuột. - SDL_QUIT event được bắt và throw QuitException để thoát game đúng cách.

II. Chi tiết cài đặt:

1. Tư duy thiết kế:

- Luồng tuần tự của Engine được giữ nguyên từ Level 1 về cấu trúc, nhưng được trừu tượng hoá thông qua interface. Điểm mấu chốt ở bài lần này là Engine không gọi trực tiếp Terminal hay SDL - nó chỉ gọi thông qua hai interface trừu tượng `I_Renderer*` và `I_Interaction*` được truyền từ `main()`. Toàn bộ sự khác biệt giữa Terminal và SDL tách biệt hoàn toàn khỏi Engine. Về cơ bản, cấu trúc của Engine vẫn bao gồm:

+ `startGame()` gọi tuần tự: title → chọn size → chọn goal → chọn mode → chọn bot level → khởi tạo board.

+ `playGame()` là game loop chính: render board → lấy nước đi (bot hoặc người) → make move → kiểm tra kết quả (`checkWin`, `checkDraw`) → chuyển lượt.

+ `endGame()` hiển thị kết quả, highlight win line và chờ người dùng tương tác lần cuối.

- Tại sao lại lựa chọn thiết kế này?

+ Game theo lượt có một đặc điểm quan trọng: trạng thái chỉ thay đổi khi có input từ người chơi hoặc bot. Giữa hai lượt, không có gì cần cập nhật. Vì vậy, luồng tuần tự khai thác đúng tính chất này: chương trình chỉ làm việc khi thực sự có input, còn lại nó chờ người dùng tương tác. Với Terminal, đó là `cin >>`. Với SDL, đó là `SDL_WaitEvent()`. Cả hai đều không tiêu tốn tài nguyên máy tính trong lúc chờ.

+ Trừu tượng hóa cơ chế xử lý bất đồng bộ thông qua Interface là giải pháp kiến trúc cốt lõi nhằm tách biệt Game Engine khỏi Terminal và SDL. Thay vì lạm dụng các câu lệnh rẽ nhánh `if-else` trong Engine để xử lý sự khác biệt về cấu trúc input, hệ thống đóng gói toàn bộ logic phức tạp này vào các lớp thực thi kiểm soát dữ liệu. Theo đó, mỗi phương thức thuộc `SDL/Terminal Interaction` sẽ chịu trách nhiệm tự vận hành một vòng lặp sự kiện riêng biệt.

Ví dụ: Engine gọi `iInteraction` → `selectSize(&gameSetup.size)` - cùng một dòng code, hoạt động với cả hai implementation mà không cần biết bên trong thực sự sẽ làm gì.

+ Level 1 đã xây dựng luồng tuần tự này. Khi nâng lên Level 2, ta có thể giữ nguyên luồng và chỉ bọc lại các hàm bằng interface vì logic đã được kiểm chứng, chỉ cần tái cấu trúc lớp ngoài.

- Ưu và nhược điểm của thiết kế trên:

Ưu điểm	Nhược điểm
- Engine hoàn toàn độc lập với UI: thêm giao diện mới chỉ cần tạo class kế thừa interface, không phải sửa Engine - Luồng thực thi dễ đọc, dễ debug, dễ theo dõi trong log - Tách biệt các module ra, giúp dễ dàng tìm kiếm và sửa chữa từng module nếu cần một cách nhanh chóng, tiện lợi.	- Trong Engine, thời gian <code>sleep_for()</code>, không có gì xử lý event – cửa sổ có thể bị treo. - Game Engine khi chạy SDL không tự quản lý event loop như Terminal Interaction mà SDL Interaction phải tự quản lý - Khó tiếp cận với người mới hơn so với Level 1, yêu cầu phải nắm vững các nguyên tắc và cú pháp phức tạp hơn của OOP cũng như thư viện đồ họa SDL

2. Giao diện đồ họa:

(1) Hiển thị:

- Thay vì sử dụng tài nguyên hình ảnh có sẵn, tôi chọn giải pháp dựng giao diện trực tiếp từ các khối hình học cơ bản. Toàn bộ thành phần đồ họa được cấu thành hoàn toàn từ các hàm cốt lõi của thư viện SDL2 và SDL2_TTF như:
 - + SDL_RenderFillRect / SDL_RenderDrawRect: Đảm nhiệm dựng các khối vuông như ô cờ, hệ thống nút bấm và các khung điều khiển.
 - + SDL_RenderDrawLine: Đóng vai trò phân tách và dựng lưới cho bàn cờ.
 - + SDL_ttf (TTF_RenderText_Blended): Xử lý hiển thị các ký tự chữ hoặc số trên giao diện.
 - + Chỉ dùng SDL_image để trang trí background các menu chọn không cần cập nhật real time, chỉ hiển thị một lần như titlescreen hay chọn chế độ chơi, độ khó bot.

- Lý do chọn hình thức hiển thị trên:

- + Phù hợp với bản chất game. TicTacToe là game trừu tượng - board lưới, ký hiệu X/O. Chỉ cần những hình học đơn giản để thể hiện đúng bản chất đó, không cần đồ họa phức tạp.
- + Dễ scale theo kích thước board. Nếu sử dụng hình ảnh vẽ sẽ phải tính toán nhiều bước khá phức tạp.
- + Mỗi lần render một frame chỉ trong vài milli giây, vẫn có thể đem đến cho người chơi trải nghiệm mượt mà thời gian thực.

(2) Tương tác:

- Mỗi hàm trong SDLInteraction tự chạy vòng lặp while() riêng, xử lý event phù hợp với từng màn hình. Khi Engine gọi một hàm interaction, hàm đó sẽ chờ và liên tục gọi SDL_WaitEvent cho đến khi nhận được input hợp lệ rồi mới trả kết quả về cho Engine.
- Các tương tác được hỗ trợ:
 - + Nhập số (chọn size, goal): Người dùng gõ từng chữ số trên bàn phím. Mỗi phím được append vào inputBuffer bên trong SDLRenderer và hiển thị real-time lên màn hình. Nhấn Backspace để xóa ký tự cuối, nhấn Enter để xác nhận.
 - + Chọn chế độ chơi và bot level: Hỗ trợ cả hai cách — click chuột vào ô trên màn hình, hoặc nhấn phím số 1/2/3. Tọa độ click được kiểm tra có nằm trong vùng của từng lựa chọn không, nếu không nằm trong vùng thì người chơi click tiếp hoặc bấm số để chọn.
 - + Đặt quân cờ: Người dùng click chuột vào ô trên board. Tọa độ pixel được chuyển về (row, col) theo công thức
 - + Thoát game: SDL_QUIT (bấm X cửa sổ) được kiểm tra ở mọi vòng lặp event thông qua hàm waitForQuit(). Khi phát hiện, hàm này ném QuitException được bắt ở main() để dọn dẹp tài nguyên để thoát game đúng cách.
- Khó khăn lớn nhất: xây dựng ô gõ phím cập nhật realtime khi chọn size hoặc goal.
- + SDL không có cơ chế nhập chuỗi sẵn như cin. Để nhập số, phải tự xây dựng hoàn toàn:

- SDLRenderer giữ một inputBuffer (string) lưu ký tự đang nhập
- Người dùng nhấn phím số → appendInputBuffer(char), màn hình cập nhật ngay
- Nhấn Backspace → popInputBuffer(), xoá ký tự cuối
- Nhấn Enter → đọc buffer, chuyển sang int, validate, xoá buffer

+ Vấn đề phát sinh: để cập nhật màn hình real-time trong lúc gõ, SDLInteraction cần gọi lại SDLRenderer để hiển thị buffer. Điều này tạo ra sự phụ thuộc trực tiếp SDLInteraction → SDLRenderer thay vì qua interface I_Renderer. Đây là điểm yếu trong thiết kế khó tránh trong phạm vi bài tập - vì I_Renderer không nên có các method quản lý input buffer như appendInputBuffer hay getInputBuffer...

3. Nguyên tắc của OOP và áp dụng mẫu thiết kế:

(1) Lập trình hướng đối tượng: Trò chơi được thiết kế đã tuân thủ 4 nguyên lý quan trọng của lập trình hướng đối tượng:

- **Đóng gói (Encapsulation):** Chương trình đã thực hiện tốt việc che giấu dữ liệu nội bộ của một đối tượng, chỉ cho phép truy cập thông qua các phương thức công khai. Điều này bảo vệ tính toàn vẹn dữ liệu và giảm sự phụ thuộc giữa các phần của chương trình.

Ví dụ: Engine, Bot, SDLRenderer, ...

```
class Engine {
private:
    const RunConfig* config;      // Che giấu cấu hình
    I_Renderer* iRenderer;        // Che giấu renderer
    I_Interaction* iInteraction;  // Che giấu interaction

    GameSetup gameSetup;         // Che giấu trạng thái setup

    bool sanity_check();          // Phương thức private
public:
    Engine(const RunConfig* _config, I_Renderer* _iRenderer,
           I_Interaction* _iInteraction);
    ~Engine();
    // Các phương thức public để điều khiển game
    bool init();
    bool startGame();
    bool playGame();
    void endGame();
    void close();
};
```

- + Tất cả dữ liệu thành viên đều được khai báo là private.
- + Người dùng Engine không thể trực tiếp thay đổi config, iRenderer, hay iInteraction.
- + Chỉ có thể tương tác thông qua các phương thức public: init(), startGame(), playGame(), endGame(), close().

```

class Bot {
protected:
    const BotLevel level;           // Mức độ của bot
    const char symbol;              // Ký hiệu ('X' hoặc 'O')
    const char op_symbol;           // Ký hiệu đối thủ
public:
    Bot(const BotLevel& _level, const char& _symbol);
    virtual ~Bot();
    virtual pII getMove(char board[][BOARD_N_MAX], const int size,
        const int goal) = 0;
};

```

+ Các thuộc tính level, symbol, op_symbol được khai báo là protected.

+ Chỉ các lớp con của Bot mới có thể truy cập trực tiếp.

+ Các lớp con không thể sửa đổi được (const).

→ Đảm bảo tính nhất quán: một Bot luôn có symbol và level xác định.

- **Kế thừa (Inheritance):** Chương trình cho phép một lớp con kế thừa các thuộc tính và phương thức từ lớp cha.

Ví dụ: BotLevel, Renderer

```

class BotLevel1 : public Bot { // Kế thừa từ Bot
protected:
    pII random_pick(char board[][BOARD_N_MAX], const int size);
public:
    BotLevel1(const BotLevel& _level, const char& _symbol);
    ~BotLevel1() override;
    pII getMove(char board[][BOARD_N_MAX], const int size,
        const int goal) override; // Override phương thức cha
};

```

```

class BotLevel2 : public BotLevel1 { // Kế thừa từ BotLevel1
protected:
    pII heuristic_pick(char board[][BOARD_N_MAX], const int size,
        const int goal, const char botSymbol, const char playerSymbol);
public:
    BotLevel2(const BotLevel& _level, const char& _symbol);
    ~BotLevel2() override;
    pII getMove(char board[][BOARD_N_MAX], const int size,
        const int goal) override;
};

```

```

class BotLevel3 : public BotLevel2 { // Kế thừa từ BotLevel2
    // Thêm chiến lược nâng cao hơn (minimax, alpha-beta pruning, ...)
public:
    BotLevel3(const BotLevel& _level, const char& _symbol);
    ~BotLevel3() override;
    pII getMove(char board[][BOARD_N_MAX], const int size,
        const int goal) override;
};

```

+ BotLevel2 không viết lại random_pick mà gọi thẳng từ BotLevel1 làm fallback khi heuristic không tìm được nước đi (trên thực tế, trong mã nguồn của tôi thay thế cho BotLevel2 thay vì gọi random_pick thì ưu tiên ô gần trung tâm nên không gọi lại random_pick kế thừa từ BotLevel1). BotLevel3 kế thừa toàn bộ logic của BotLevel2.

```
class I_Renderer {
public:
    I_Renderer();
    virtual ~I_Renderer();
    virtual void init(const RunConfig& config) = 0;
    virtual void clearScreen() = 0;
    virtual void renderBoard(const char board[][BOARD_N_MAX], const int size,
                             const int currentPlayer) = 0;
    virtual void showMove(const char board[][BOARD_N_MAX], const int size,
                          const pII move, const char symbol) = 0;
    // ... các phương thức khác
};
```

```
class TerminalRenderer : public I_Renderer {
public:
    TerminalRenderer();
    ~TerminalRenderer();
    void init(const RunConfig& config) override;
    void clearScreen() override;
    void renderBoard(const char board[][BOARD_N_MAX], const int size,
                     const int currentPlayer) override;
    void showMove(const char board[][BOARD_N_MAX], const int size,
                  const pII move, const char symbol) override;
};
```

+ Ngoài BotLevel ra, cả SDL và Terminal của Renderer và Interaction cũng được kế thừa từ interface I_Renderer và I_Interaction.

- **Tính đa hình (Polymorphism):** Chương trình cho phép gọi các phương thức khác nhau thông qua cùng một interface.

Ví dụ: Engine gọi iRenderer → displayBoard() - tại runtime là TerminalRenderer hoặc SDLRenderer tùy cờ -g, không có if/else nào trong Engine. Tương tự, bots[player] → getMove() đến đúng tới BotLevel1/2/3 tùy level được chọn. Destructor ảo trên Bot, I_Renderer, I_Interaction đảm bảo khi delete qua con trỏ thì destructor lớp con được gọi đúng.

- **Trừu tượng hóa (Abstraction):** ẩn đi chi tiết phức tạp, chỉ cung cấp interface đơn giản.

Ví dụ: I_Renderer, I_Interaction, ...

```

class I_Renderer {
public:
    I_Renderer();
    virtual ~I_Renderer();

    // Pure virtual functions - KHÔNG CÓ implementation
    virtual void init(const RunConfig& config) = 0;
    virtual void clearScreen() = 0;
    virtual void renderBoard(const char board[][BOARD_N_MAX], const int size,
                             const int currentPlayer) = 0;
    virtual void showMove(const char board[][BOARD_N_MAX], const int size,
                           const pII move, const char symbol) = 0;
    virtual void showMenu(const SelectType& selectType,
                           const std::string& context = "") = 0;
    virtual void showResult(const GameResult& result) = 0;
    virtual void showNotification(const std::string& message) = 0;
};

```

- + I_Renderer là một interface
- + Engine không cần biết Rendering là terminal hay SDL?
- + Engine chỉ cần biết: Gọi renderer → renderBoard() để vẽ bàn cờ

(2) Thiết kế cân bằng hai giao diện:

- Kiến trúc game được thiết kế để hỗ trợ cả Terminal và GUI một cách cân bằng. Hai giao diện có thể hoán đổi được thông qua command-line flag.

- Thứ cốt lõi giúp hài hoà - interface trừu tượng

+ I_Renderer và I_Interaction là điểm giao tiếp duy nhất giữa Engine và UI. Engine không có bất kỳ dòng code nào biết đến Terminal hay SDL - nó chỉ gọi method trừu tượng. Toàn bộ sự khác biệt giữa hai giao diện được đẩy xuống bên trong từng implementation, Engine không cần biết và không cần thay đổi. Việc chạy SDL hay Terminal sẽ do main() quyết định

+ Vấn đề chưa hoàn toàn cân bằng: Có một điểm thiết kế phá vỡ sự đối xứng: SDLInteraction cần tham chiếu trực tiếp tới SDLRenderer* để cập nhật inputBuffer realtime trong khi người dùng đang gõ. Đây là sự phụ thuộc giữa hai class cụ thể - không qua interface.

(3) Đánh giá mức độ tuân thủ SOLID:

- **S - Single Responsibility Principle:** Tuân thủ tốt. Mỗi class chỉ có một trách nhiệm rõ ràng: Engine điều phối luồng game, Logic xử lý luật chơi, TerminalRenderer/SDLRenderer chỉ hiển thị, TerminalInteraction/SDLInteraction chỉ nhận input, BotFactory chỉ tạo bot, ...

- **O - Open/Closed Principle:** Tuân thủ một phần.

+ Tuân thủ ở hệ thống Bot và UI: thêm BotLevel4 chỉ cần tạo class mới kế thừa BotLevel2 và thêm một case trong BotFactory, không sửa Engine. Thêm giao diện mới chỉ cần implement I_Renderer và I_Interaction, không phải sửa Engine.

+ Vi phạm ở showSelectMenu() trong cả TerminalRenderer và SDLRenderer: dùng switch-case trên SelectType. Mỗi lần thêm màn hình mới phải vào sửa trực tiếp hàm này.

- **L - Liskov Substitution Principle:** Tuân thủ tốt. TerminalRenderer và SDLRenderer thay thế hoàn toàn I_Renderer mà không làm sai hành vi Engine - Engine gọi displayBoard(), showResult() và nhận kết quả đúng dù là implementation nào. Tương tự BotLevel2 thay thế BotLevel1 hoạt động đúng - vẫn trả về nước đi hợp lệ, chỉ là đưa ra nước đi thông minh hơn...

- **I - Interface Segregation Principle:** Tuân thủ tốt. I_Renderer và I_Interaction được tách thành hai interface riêng biệt, không gộp chung, không liên quan đến nhau.

- **D - Dependency Inversion Principle:** Tuân thủ một phần.

+ Tuân thủ ở Engine: phụ thuộc hoàn toàn vào I_Renderer và I_Interaction - hai interface trừu tượng.

+ Vi phạm ở SDLInteraction: phụ thuộc trực tiếp vào SDLRenderer

Lý do vi phạm: SDLInteraction cần gọi appendInputBuffer(), getInputBuffer() - những method chỉ có trên SDLRenderer. Nếu đổi thành I_Renderer thì không gọi được các method này. Nếu thêm các method đó vào I_Renderer thì vi phạm ISP vì TerminalRenderer không cần chúng. Đây là vấn đề giữa DIP và ISP và tôi chưa có lời giải hoàn hảo trong thiết kế hiện tại.

(4) Sử dụng mẫu thiết kế: Chương trình của tôi đã sử dụng rất nhiều mẫu thiết kế lập trình hướng đối tượng nhưng dưới đây là 3 mẫu thiết kế đáng chú ý nhất.

- **Factory Method:** BotFactory: Tách logic khởi tạo đối tượng ra khỏi nơi sử dụng. Thay vì Engine tự new BotLevel1/2/3, việc đó được uỷ quyền cho BotFactory.

```
Bot* BotFactory::createBot(BotLevel level, const char& symbol) {
    // TODO: implement factory logic

    switch (level) {
        case BotLevel::EASY:
            // TODO: tạo bot level EASY (ví dụ BotLevel1)
            return new BotLevel1(level, symbol);

        case BotLevel::MEDIUM:
            // TODO: tạo bot level MEDIUM (ví dụ BotLevel2)
            return new BotLevel2(level, symbol);

        case BotLevel::HARD:
            // TODO: tạo bot level HARD (ví dụ BotLevel3)
            return new BotLevel3(level, symbol);

        default:
            // TODO: xử lý level không hợp lệ
            return nullptr;
    }
}
```

- **Dependency Injection:** Engine: Thay vì tự tạo dependency bên trong, Engine nhận chúng từ bên ngoài qua constructor. main() quyết định inject implementation nào.

```
I_Renderer* iRenderer = new SDLRenderer();
I_Interaction* iInteraction = new SDLInteraction();

// Inject vào Engine qua constructor
Engine* engine = new Engine(&config, iRenderer, iInteraction);

Engine::Engine(const RunConfig* _config,
               I_Renderer* _iRenderer,
               I_Interaction* _iInteraction) {
    config = _config;
    iRenderer = _iRenderer;
    iInteraction = _iInteraction;
}
```

- **Singleton:** Singleton đảm bảo chỉ có một instance duy nhất của một class/module trong toàn bộ chương trình, và có thể truy cập từ bất kỳ đâu.

Logger được implement dưới dạng namespace với toàn bộ biến là inline static - tức là chỉ tồn tại một bản duy nhất trong toàn chương trình, không thể tạo thêm instance:

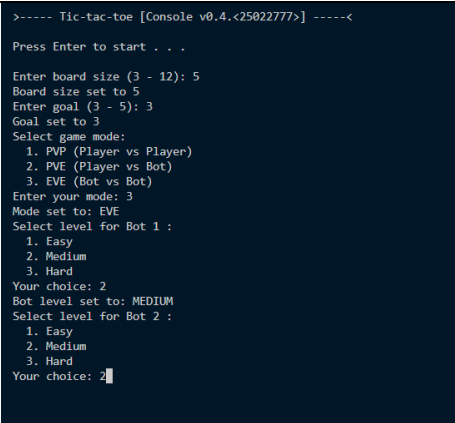
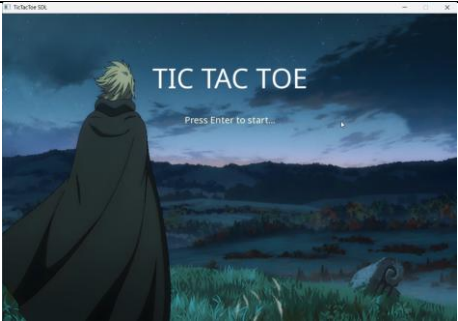
```
namespace Logger {
    inline static Level min_level = Level::INFO; // duy nhất toàn chương trình
    inline static std::ofstream log_file; // duy nhất toàn chương trình
    inline static bool write_to_file = false; // duy nhất toàn chương trình
    inline static bool is_judge_mode = false; // duy nhất toàn chương trình

    void init(...); // khởi tạo một lần duy nhất
    void log(...); // gọi từ bất kỳ đâu
    void close();
}
```

III. Kết quả thực nghiệm:

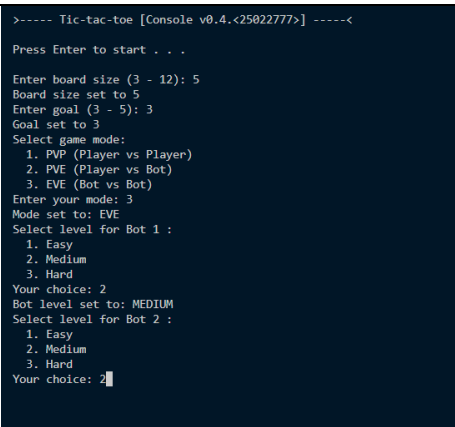
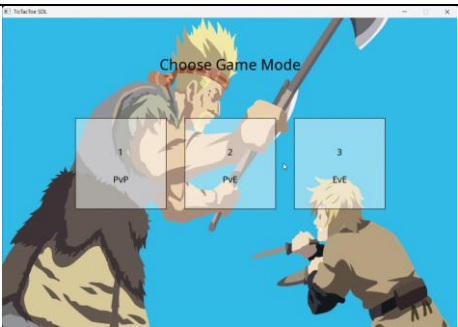
- Dưới đây là sự khác biệt giữa màn hình chơi với Terminal và SDL

1 - Màn hình khởi động

	
Terminal: menu nhập text bằng cin >>	SDL: title screen với background ảnh Vinland Saga

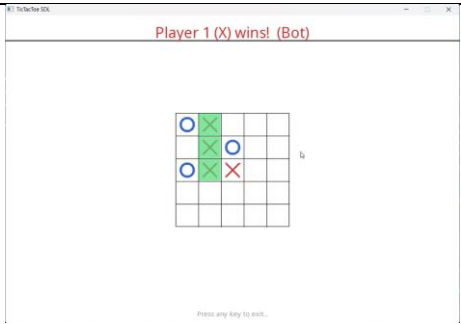
Nhận xét: Terminal hiển thị tuần tự qua cout và chờ cin. SDL có màn hình title riêng với background ảnh; người dùng bấm Enter hoặc click để tiếp tục. Cùng một luồng Engine nhưng trải nghiệm người dùng hoàn toàn khác nhau.

2 - Màn hình chọn chế độ chơi

	
Terminal: in lựa chọn, nhập số bằng cin	SDL: nút bấm - click chuột hoặc nhấn phím 1/2/3 để chọn

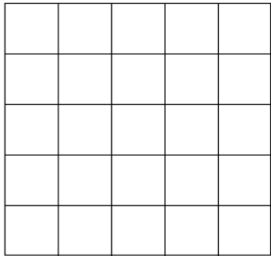
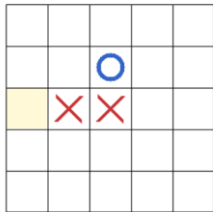
Nhận xét: Cùng gọi selectGameMode() nhưng TerminalInteraction đơn giản là cin >> choice. SDLInteraction chạy event loop riêng, phát hiện SDL_MOUSEBUTTONDOWN trong vùng nút hoặc SDL_KEYDOWN phím 1-3. Engine không thay đổi một dòng nào nhưng vẫn hoạt động tốt ở cả hai màn hình Terminal và SDL.

3 - Bàn cờ trong trận đấu

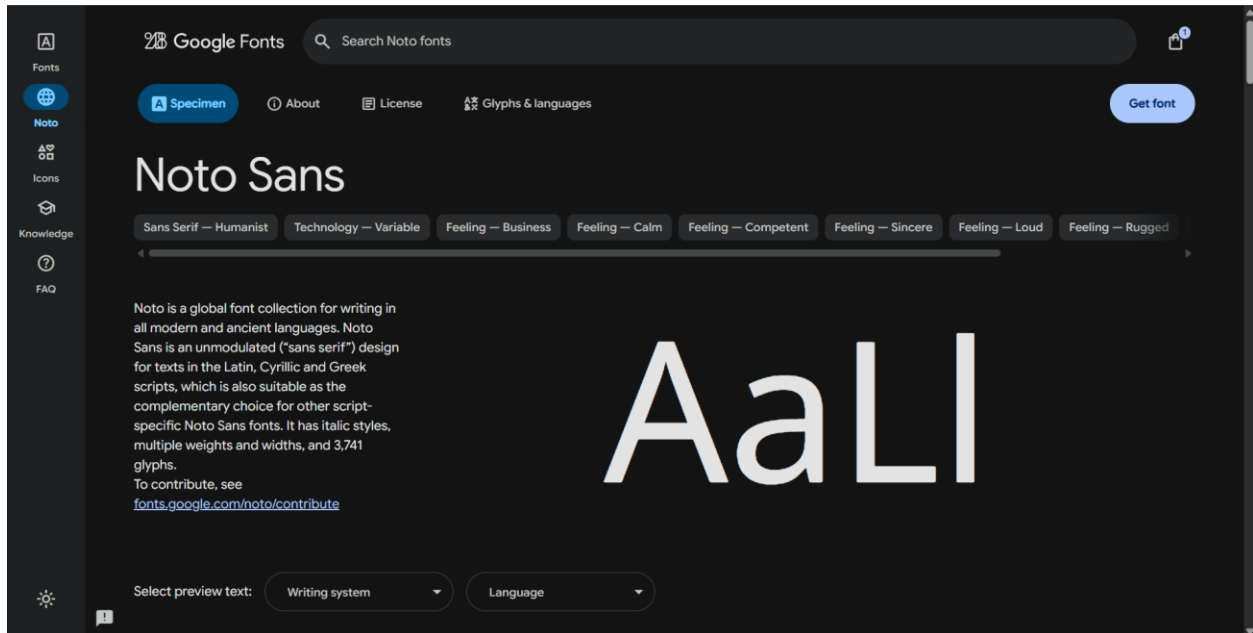
<pre>0 1 2 3 4 ----- 0 - - - - 1 - - 0 - 2 0 X X - 3 - - - - 4 - - - - Player 1 (Bot) is thinking... Made move at (1, 1)</pre>	
Terminal: vẽ bằng ASCII sử dụng cout	SDL: vẽ bằng thư viện đồ họa SDL

Nhận xét: displayBoard() trong TerminalRenderer in ký tự +, -, |, ... để vẽ bàn cờ. SDLRenderer vẽ lưới và ký hiệu bằng các hàm trong SDL. Kích thước tự scale theo board size được chọn - nhờ thiết kế tính cellSize động, không cần sửa code vẽ.

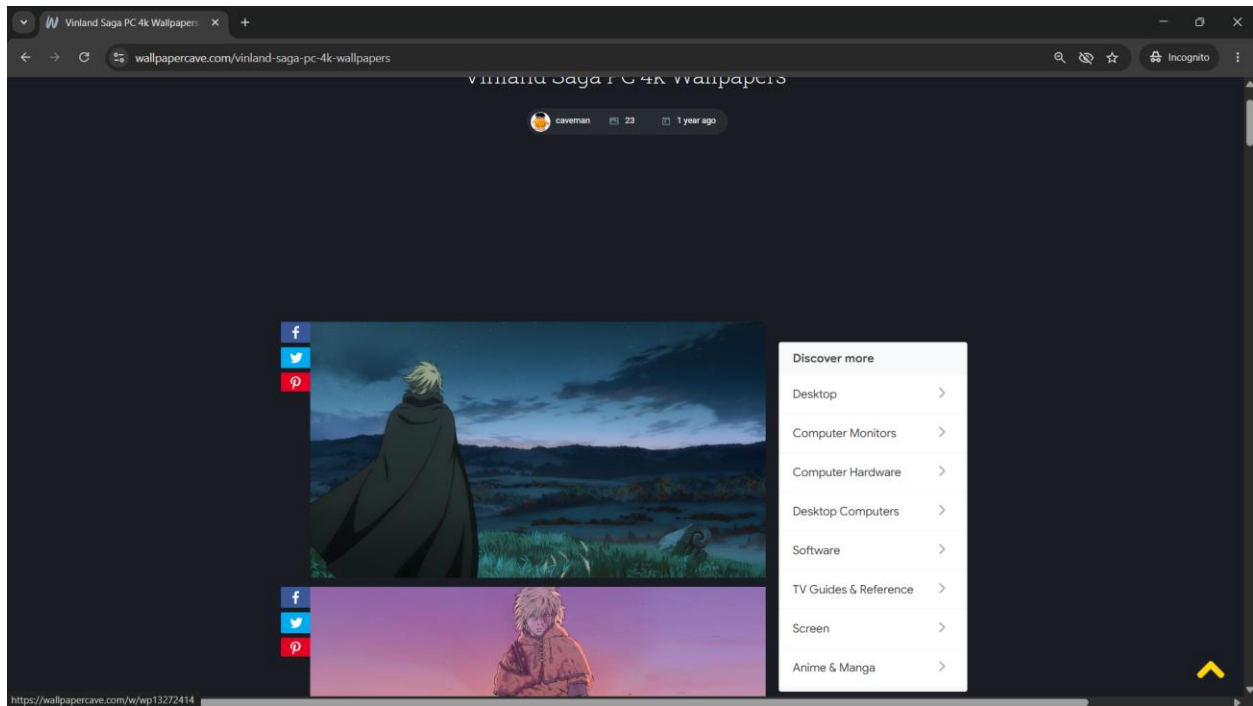
4 - Nước đi người chơi và bot trong SDL

Click to make your move... 	Player 2 (O)'s turn (Bot) 
Click chuột để đặt quân	Bot đặt quân tự động

- Tài nguyên game được sử dụng:
- + Font chữ: Noto Sans (Google Font)

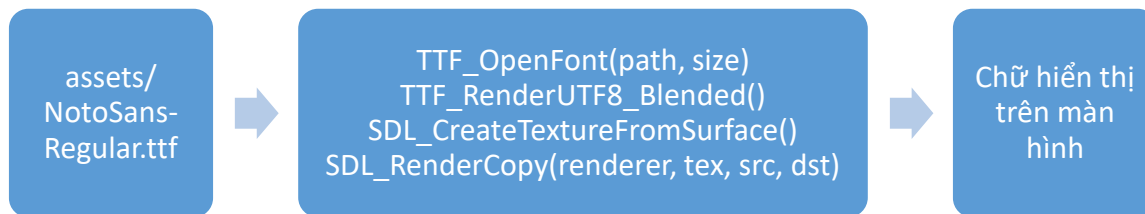


- + Ảnh trang trí các menu: <https://wallpapercave.com/vinland-saga-pc-4k-wallpapers>

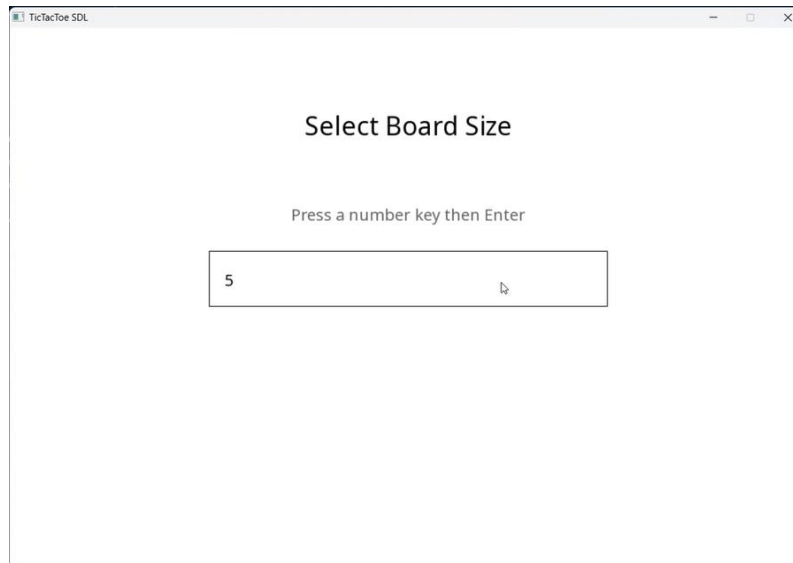


- Cách tài nguyên được hiển thị lên màn hình game:

+ Font chữ:

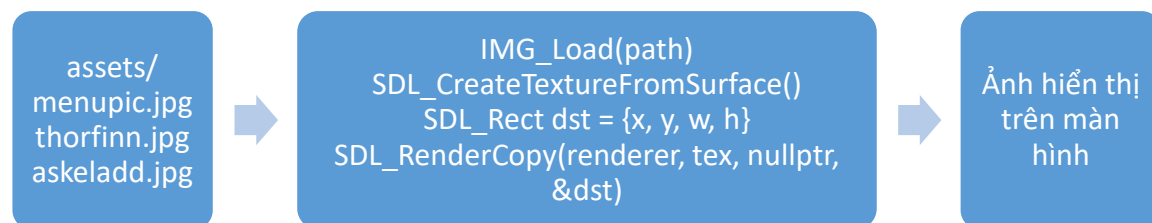


Kết quả: Menu chọn kích thước bàn cờ, goal sử dụng font chữ Noto Sans:



Chữ trên màn hình được render qua TTF_RenderUTF8_Blended() từ NotoSans-Regular.ttf

+ Ảnh:



Raw asset - thorfinn.jpg và kết quả sau khi render - menu chọn bot level:



- Trích dẫn log và phân tích:

```
1 Tic-tac-toe Game (Version: 0.4.<25022777> by balueung.87)
2 -----
3 [INFO] - [2026-05-22 10:36:39] Logger initialized!
4 [DEBUG] - [2026-05-22 10:36:39] RunConfig {
5     interactive: true
6     judge_mode: false
7     input_file: ""
8     to_file: true
9     log_file: "log.txt"
10    verbose_flag: true
11    gui_flag: false
12    is_help: false
13 }
14 [INFO] - [2026-05-22 10:36:39] Terminal Renderer initialized!
15 [INFO] - [2026-05-22 10:36:39] Terminal Interaction initialized!
16 [INFO] - [2026-05-22 10:36:39] Infra initialized!
17 [INFO] - [2026-05-22 10:36:39] Engine initializing . . .
18 [INFO] - [2026-05-22 10:36:39] Engine initialized!
19 [INFO] - [2026-05-22 10:36:39] [Engine] Starting game . . .
20 [DEBUG] - [2026-05-22 10:36:42] user input 'size' = 5
21 [DEBUG] - [2026-05-22 10:36:45] user input 'goal' = 3
22 [DEBUG] - [2026-05-22 10:36:49] user input 'game mode' = EVE
23 [DEBUG] - [2026-05-22 10:36:50] user input 'bot level[0]' = EASY
24 [DEBUG] - [2026-05-22 10:36:51] user input 'bot level[1]' = MEDIUM
25 [INFO] - [2026-05-22 10:36:51] Board initialized!
26 [INFO] - [2026-05-22 10:36:51] [Engine] Game started!
27 [INFO] - [2026-05-22 10:36:51] [Engine] Playing game . . .
28 [DEBUG] - [2026-05-22 10:36:51] [Engine] starting turn #0
29 [DEBUG] - [2026-05-22 10:36:51] execution time of [bot#0->getMove()] = 2.6e-06s
30 [DEBUG] - [2026-05-22 10:36:51] player 1 make move to (1, 2)
31 [DEBUG] - [2026-05-22 10:36:52] [Engine] turn done!
32 [DEBUG] - [2026-05-22 10:36:52] [Engine] starting turn #1
33 [DEBUG] - [2026-05-22 10:36:52] execution time of [bot#1->getMove()] = 6.5e-06s
34 [DEBUG] - [2026-05-22 10:36:52] player 2 make move to (2, 2)
35 [DEBUG] - [2026-05-22 10:36:54] [Engine] turn done!
36 [DEBUG] - [2026-05-22 10:36:54] [Engine] starting turn #2
37 [DEBUG] - [2026-05-22 10:36:54] execution time of [bot#0->getMove()] = 1.5e-06s
38 [DEBUG] - [2026-05-22 10:36:54] player 1 make move to (1, 4)
39 [DEBUG] - [2026-05-22 10:36:55] [Engine] turn done!
40 [DEBUG] - [2026-05-22 10:36:55] [Engine] starting turn #3
41 [DEBUG] - [2026-05-22 10:36:55] execution time of [bot#1->getMove()] = 5.9e-06s
42 [DEBUG] - [2026-05-22 10:36:55] player 2 make move to (1, 3)
43 [DEBUG] - [2026-05-22 10:36:57] [Engine] turn done!
44 [DEBUG] - [2026-05-22 10:36:57] [Engine] starting turn #4
45 [DEBUG] - [2026-05-22 10:36:57] execution time of [bot#0->getMove()] = 1.7e-06s
46 [DEBUG] - [2026-05-22 10:36:57] player 1 make move to (4, 1)
47 [DEBUG] - [2026-05-22 10:36:58] [Engine] turn done!
48 [DEBUG] - [2026-05-22 10:36:58] [Engine] starting turn #5
49 [DEBUG] - [2026-05-22 10:36:58] execution time of [bot#1->getMove()] = 1.85e-05s
50 [DEBUG] - [2026-05-22 10:36:58] player 2 make move to (0, 4)
51 [INFO] - [2026-05-22 10:37:00] Player 2 (0) wins at turn 6
52 [DEBUG] - [2026-05-22 10:37:00] [Engine] turn done!
53 [INFO] - [2026-05-22 10:37:00] [Engine] Game done!
54 [INFO] - [2026-05-22 10:37:00] [Engine] Ending game . . .
55 [DEBUG] - [2026-05-22 10:37:00] [WinLine] cells: (0, 4) (1, 3) (2, 2)
56 [DEBUG] - [2026-05-22 10:37:00] after 6 turns
57 [DEBUG] - [2026-05-22 10:37:00] game end with result: player 2 (bot) win!
58 [INFO] - [2026-05-22 10:37:00] [Engine] Game ended!
59 [INFO] - [2026-05-22 10:37:00] Engine closing . . .
60 [INFO] - [2026-05-22 10:37:00] Engine closed!
61 [INFO] - [2026-05-22 10:37:00] Engine destroyed!
62 [INFO] - [2026-05-22 10:37:00] Infra destroyed!
63 [INFO] - [2026-05-22 10:37:00] Logger closing . . .
64
```

log_default.txt (Chạy Terminal)

```
1 Tic-tac-toe Game (Version: 0.4.<25022777> by balueung.87)
2 -----
3 [INFO] - [2026-05-22 10:49:04] Logger initialized!
4 [DEBUG] - [2026-05-22 10:49:04] RunConfig {
5     interactive: true
6     judge_mode: false
7     input_file: ""
8     to_file: true
9     log_file: "log.txt"
10    verbose_flag: true
11    gui_flag: true
12    is_help: false
13 }
14 [INFO] - [2026-05-22 10:49:04] SDL Renderer initialized!
15 [INFO] - [2026-05-22 10:49:04] SDL Interaction initialized!
16 [INFO] - [2026-05-22 10:49:04] Infra initialized!
17 [INFO] - [2026-05-22 10:49:04] Engine initializing . . .
18 [INFO] - [2026-05-22 10:49:04] Engine initialized!
19 [INFO] - [2026-05-22 10:49:04] [Engine] Starting game . . .
20 [DEBUG] - [2026-05-22 10:49:08] user input 'size' = 5
21 [DEBUG] - [2026-05-22 10:49:13] user input 'goal' = 3
22 [DEBUG] - [2026-05-22 10:49:15] user input 'game mode' = EVE
23 [DEBUG] - [2026-05-22 10:49:16] user input 'bot level[0]' = MEDIUM
24 [DEBUG] - [2026-05-22 10:49:17] user input 'bot level[1]' = MEDIUM
25 [INFO] - [2026-05-22 10:49:17] Board initialized!
26 [INFO] - [2026-05-22 10:49:17] [Engine] Game started!
27 [INFO] - [2026-05-22 10:49:17] [Engine] Playing game . . .
28 [DEBUG] - [2026-05-22 10:49:17] [Engine] starting turn #0
29 [DEBUG] - [2026-05-22 10:49:17] execution time of [bot#0->getMove()] = 1.83e-05s
30 [DEBUG] - [2026-05-22 10:49:17] player 1 make move to (2, 2)
31 [DEBUG] - [2026-05-22 10:49:19] [Engine] turn done!
32 [DEBUG] - [2026-05-22 10:49:19] [Engine] starting turn #1
33 [DEBUG] - [2026-05-22 10:49:19] execution time of [bot#1->getMove()] = 1.47e-05s
34 [DEBUG] - [2026-05-22 10:49:19] player 2 make move to (1, 2)
35 [DEBUG] - [2026-05-22 10:49:20] [Engine] turn done!
36 [DEBUG] - [2026-05-22 10:49:20] [Engine] starting turn #2
37 [DEBUG] - [2026-05-22 10:49:20] execution time of [bot#0->getMove()] = 6.2e-06s
38 [DEBUG] - [2026-05-22 10:49:20] player 1 make move to (2, 1)
39 [DEBUG] - [2026-05-22 10:49:22] [Engine] turn done!
40 [DEBUG] - [2026-05-22 10:49:22] [Engine] starting turn #3
41 [DEBUG] - [2026-05-22 10:49:22] execution time of [bot#1->getMove()] = 1.01e-05s
42 [DEBUG] - [2026-05-22 10:49:22] player 2 make move to (2, 0)
43 [DEBUG] - [2026-05-22 10:49:23] [Engine] turn done!
44 [DEBUG] - [2026-05-22 10:49:23] [Engine] starting turn #4
45 [DEBUG] - [2026-05-22 10:49:23] execution time of [bot#0->getMove()] = 1.13e-05s
46 [DEBUG] - [2026-05-22 10:49:23] player 1 make move to (1, 1)
47 [DEBUG] - [2026-05-22 10:49:25] [Engine] turn done!
48 [DEBUG] - [2026-05-22 10:49:25] [Engine] starting turn #5
49 [DEBUG] - [2026-05-22 10:49:25] execution time of [bot#1->getMove()] = 1.02e-05s
50 [DEBUG] - [2026-05-22 10:49:25] player 2 make move to (0, 0)
51 [DEBUG] - [2026-05-22 10:49:26] [Engine] turn done!
52 [DEBUG] - [2026-05-22 10:49:26] [Engine] starting turn #6
53 [DEBUG] - [2026-05-22 10:49:26] execution time of [bot#0->getMove()] = 2.8e-06s
54 [DEBUG] - [2026-05-22 10:49:26] player 1 make move to (0, 1)
55 [INFO] - [2026-05-22 10:49:28] Player 1 (2) wins at turn 7
56 [DEBUG] - [2026-05-22 10:49:28] [Engine] turn done!
57 [INFO] - [2026-05-22 10:49:28] [Engine] Game done!
58 [INFO] - [2026-05-22 10:49:28] [Engine] Ending game . . .
59 [DEBUG] - [2026-05-22 10:49:28] [WinLine] cells: (0, 1) (1, 1) (2, 1)
60 [DEBUG] - [2026-05-22 10:52:57] after 7 turns
61 [DEBUG] - [2026-05-22 10:52:57] game end with result: player 1 (bot) win!
62 [INFO] - [2026-05-22 10:52:57] [Engine] Game ended!
63 [INFO] - [2026-05-22 10:52:57] Engine closing . . .
64 [INFO] - [2026-05-22 10:52:57] Engine closed!
65 [INFO] - [2026-05-22 10:52:57] Engine destroyed!
66 [INFO] - [2026-05-22 10:52:57] Infra destroyed!
67 [INFO] - [2026-05-22 10:52:57] Logger closing . . .
68
```

log_graphic_flag.txt (Chạy GUI)

Log	Giai đoạn	Phân tích
Logger initialized!	Khởi tạo Logger	Logger được khởi tạo đầu tiên để mọi log sau đều được ghi lại đầy đủ.
Terminal/SDL Renderer initialized! - Terminal/SDL Interaction initialized!	Khởi tạo hạ tầng	Điểm khác biệt rõ nhất giữa 2 log: dòng này cho thấy implementation nào đang chạy, SDL hay Terminal.
Engine initializing... Engine initialized!	Khởi tạo Engine	Engine nhận I_Renderer* và I_Interaction* được truyền từ main().
[Engine] Starting game Board initialized! [Engine] Game started!	startGame()	Thực hiện luồng tuần tự: title → size → goal → mode → bot level → init board.
Turn → Make move → Execution time (bot) → Game done	playGame()	Trả về thực hiện thông tin trong game bao gồm nước đi được chọn, lượt nào, thời gian thực thi (nếu là bot), kết quả chung cuộc
Engine closing → closed → destroyed ...→ Logger closing...	close()	Dọn dẹp đúng thứ tự: Engine → Infra → Logger. Thiếu bất kỳ bước nào → memory leak hoặc resource không được giải phóng.

→ Sự khác nhau giữa giao diện UI và DevTool:

UI: Là lớp bề nổi sử dụng thư viện đồ họa SDL hoặc Terminal. UI có nhiệm vụ chuyển hóa các trạng thái trừu tượng của ma trận game thành hình ảnh trực quan. Người chơi chỉ tương tác thông qua chuột/bàn phím trên UI này mà không hề biết các tính toán ngầm ở dưới đó.

DevTool: Là lớp lõi kỹ thuật chạy ngầm. DevTool ghi lại dòng chảy và trạng thái chương trình theo mốc thời gian thực tế. Nó đóng vai trò kiểm soát, giám sát tài nguyên và hỗ trợ lập trình viên debug khi UI xảy ra sự cố.

IV. Nhận xét và tổng kết bài làm:

1. Về phong cách lập trình:

Điểm mạnh	Điểm yếu
- Khả năng mở rộng mà không phá vỡ code cũ. - Tách biệt trách nhiệm rõ ràng.	- Chi phí thiết kế ban đầu cao. Phải dành phần lớn thời gian thiết kế và hiểu được kiến trúc chương trình mới có thể tạo ra interface, tính kế thừa, ...

- Liệu với cách thiết kế này có dễ bảo trì khi trò chơi trở nên phức tạp hơn?

+ Khả năng bảo trì hoàn toàn có thể được đảm bảo nếu kiến trúc hệ thống ban đầu được thiết kế một cách chuẩn chỉnh. Khi các Interface được định nghĩa rõ ràng ngay từ đầu, việc mở rộng tính năng về sau sẽ trở nên dễ dàng: lập trình viên chỉ cần khởi tạo các lớp mới, thực thi các phương thức của Interface. Thiết kế này giúp ta có thể dễ dàng tích hợp thêm các loại Bot hoặc các màn hình giao diện mới mà không cần phải đọc hiểu hay can thiệp vào toàn bộ mã nguồn sẵn có.

+ Ngược lại, nếu thiết kế không tốt từ giai đoạn đầu - Interface quá cồng kềnh, các lớp vi phạm các nguyên tắc OOP sẽ biến chương trình thành một mớ hỗn độn khó bảo trì hơn cả procedural code. Lúc này, có thể bất kỳ một thay đổi nhỏ nào cũng đòi hỏi ta phải rà soát lại toàn bộ cây kế thừa và các luồng phụ thuộc...

- Có phải dự án nào cũng phù hợp với OOP?

Dù OOP là mô hình cực kỳ mạnh mẽ và phổ biến, việc lạm dụng nó vào những dự án không phù hợp sẽ gây ra sự cồng kềnh, lãng phí tài nguyên và làm chậm tốc độ phát triển.

+ OOP phù hợp khi hệ thống có nhiều thực thể với hành vi khác nhau nhưng cùng interface - ví dụ như game này, có nhiều loại bot, nhiều loại UI... Khi đó kế thừa và đa hình thực sự có giá trị. Hoặc những dự án cần bảo trì lâu dài, cần tính kế thừa, đa hình để mở rộng quy mô mà không phá vỡ kiến trúc ban đầu

+ OOP không phù hợp với các tác vụ đơn giản, tuyến tính vì có thể gây lãng phí.

2. Về giao diện người dùng:

- Những điều thú vị trong quá trình phát triển:

+ Việc thành công dựng nên menu của game bàn cờ chỉ bằng việc vẽ bằng các pixel thực sự là một trải nghiệm khá mới mẻ và thú vị. Giờ đây game không chỉ tương tác thuần bằng bàn phím và chỉ hiện bàn cờ dựng từ ASCII trên Terminal mà có thể nhìn thấy đồ họa có màu sắc và tương tác được bằng chuột trên cửa sổ Window.

+ Phần thú vị thứ hai là tự xây dựng text input system từ đầu. SDL không có cin - phải tự quản lý từng phím gõ, append vào buffer, hiển thị real-time, xử lý Backspace. Tuy phức tạp nhưng khi nghĩ ra và triển khai được thì thấy rất vui khi hoàn thành.

- Khó khăn trong quá trình cài đặt:

+ Khó khăn lớn nhất là sự khác biệt mô hình giữa Terminal và SDL. Terminal thì tương tác và hiển thị rất đơn giản nhưng SDL thì khác hoàn toàn. Hơn nữa, đây là lần đầu tôi được tiếp xúc và triển khai một game sử dụng SDL nên cũng gặp khá nhiều khó khăn.

- + Trong bài tập lần này, tôi gặp khó khăn nhiều nhất trong việc suy nghĩ về việc SDLInteraction và SDLRenderer nên hoạt động và kết hợp như nào để khi Engine gọi thì cả SDL và Terminal hoạt động tốt vì luồng thực thi là giữ nguyên trong Engine.
- + Ngoài ra việc tính toán tọa độ để xây dựng Render và Interaction cũng là một phần khá đau đầu khi mọi tính toán cần phải chuẩn xác để game hoạt động theo mong muốn.

3. Về tổ chức mã nguồn: Đánh giá phương pháp tổ chức mã nguồn đa tệp so với đơn tệp

Tiêu chí	Tiếp cận đa tệp	Tiếp cận đơn tệp
Ưu điểm	- Dễ bảo trì: Code được chia theo module, lỗi ở đâu chỉ cần sửa file đó. - Dễ mở rộng: Thêm tính năng mới chỉ cần tạo file mới, không sợ làm hỏng code cũ. - Thích hợp làm việc nhóm: Nhiều người có thể code cùng lúc trên các file khác nhau mà không bị xung đột mã nguồn.	- Ngắn gọn, nhanh: Phù hợp cho bài tập nhỏ, script ngắn. - Tiện lợi ban đầu: Không cần cấu hình liên kết file (include), không cần thiết kế kiến trúc phức tạp. - Trình biên dịch chỉ cần xử lý một file duy nhất.
Nhược điểm	- Phức tạp lúc đầu: Đòi hỏi tư duy thiết kế kiến trúc tốt, yêu cầu hiểu sâu về lập trình hướng đối tượng và các nguyên tắc, mẫu thiết kế. - Quản lý liên kết: Phải kiểm soát chặt chẽ việc #include giữa các file để tránh lỗi.	- Đối với bài tập, dự án quy mô lớn: Code dài, khó tìm kiếm lỗi và debug. - Có thể gây xung đột file nếu làm việc nhóm vì mọi người đều sửa trên một file. - Các hàm phụ thuộc chặt chẽ: Sửa một hàm có thể làm đổ vỡ toàn bộ logic.

- Khi nào nên dùng đa tệp?
- + Nên dùng khi dự án có nhiều module độc lập, nhiều người cùng làm, hoặc dự kiến mở rộng lâu dài.
- + Không cần thiết cho dự án nhỏ, tool một lần dùng - lúc đó một file sẽ đơn giản và nhanh hơn.

4. Về thiết kế chương trình: Nhìn nhận về vai trò của con người và công cụ AI trong một quy trình phát triển phần mềm? Liệu con người có thể bị thay thế? Vấn đề của các công cụ AI hiện tại là gì?

- Trải nghiệm từ bài tập lớn này đã giúp tôi nhận ra một ranh giới rõ ràng giữa Coding và Designing. Đây là cơ sở để đánh giá lại mối quan hệ giữa con người và các công cụ AI trong quy trình phát triển phần mềm hiện đại.

- Vai trò của Con người và AI trong quy trình phát triển:

- + AI là người viết code: Các công cụ AI hiện nay như Claude hay Gemini thể hiện khả năng

vượt trội trong việc coding, debugging hay tối ưu hóa các hàm. AI đóng vai trò như một trợ lý lập trình tốc độ cao, giúp tự động hóa các công việc mang tính lặp đi lặp lại.

+ Con người là kiến trúc sư: Vai trò cốt lõi của con người nằm ở giai đoạn thiết kế. Đó là việc phân tích yêu cầu bài toán từ đó tổ chức hệ thống sao cho hợp lý. Đây là những tư duy chiến lược mà AI không thể tự quyết định nếu không có sự định hướng của con người.

- Liệu AI có thể thay thế con người?

+ AI có thể thay thế phần nào những lập trình viên chỉ biết gõ code, nhưng không thể thay thế một Kỹ sư phần mềm (Software Engineer).

+ Bản chất của việc phát triển phần mềm là giải quyết vấn đề và thiết kế hệ thống. Nếu con người đưa ra một bản thiết kế sai, AI sẽ chỉ sinh ra một đồng mã nguồn rác có thể chạy nhanh hơn nhưng lại rất khó bảo trì. Con người vẫn giữ tư duy cốt lõi và chịu trách nhiệm cao nhất.

- Những vấn đề của các công cụ AI hiện nay?

+ Thiếu context: Khi đối mặt với một bài toán quy mô lớn, AI rất dễ bị lạc lối, không đánh giá được tình hình nên việc coding trở nên khá khó khăn.

+ Không có tư duy phản biện: AI chỉ tổng hợp dựa trên dữ liệu có sẵn nên không thể tự đặt những câu hỏi để tối ưu mã nguồn.

5. Bài học rút ra:

- Khó khăn lớn nhất trong toàn bộ dự án này không nằm ở kỹ năng lập trình mà nằm ở tư duy thiết kế hệ thống. Cụ thể là bài toán: Thiết kế một luồng thực thi cân bằng, có khả năng điều hòa cả giao diện Terminal lẫn đồ họa SDL.

- Bản chất vấn đề: Lập trình Terminal tuần tự với cơ chế chặn luồng với cin rất đơn giản; trong khi đó, SDL lại vận hành theo mô hình event-driven. Thử thách đặt ra là phải định nghĩa một Interface chung, ép buộc cả hai mô hình có bản chất trái ngược nhau này cùng tuân thủ.

- Cách vượt qua là phải chấp nhận rằng không thể đưa ra giải pháp nào tuân thủ nguyên tắc OOP 100%, để Engine chạy được đúng với cả Terminal và SDL đã phải đánh đổi với một vài nguyên lý SOLID. Tuy nhiên điều quan trọng nhất vẫn là việc hiểu rằng mình đã đánh đổi gì và điều đó mang lại lợi ích và rủi ro gì.

V. Ghi chú sử dụng AI:

Trong bài tập lớn lần này, tôi đã sử dụng 2 mô hình AI giúp đỡ học tập là Claude, Gemini và coding agent Github Copilot tích hợp trên VSCode.

Ước tính sử dụng trong cả bài tập lần này bao gồm coding, giải thích, gợi ý ý tưởng: ~ 30%

1. Claude:

Mục đích sử dụng	Mức độ sử dụng ước tính
Gợi ý cách vẽ giao diện menu và trò chơi	~ 35 - 40%
Gợi ý cách kết hợp checkWin và getWinLine	~ 15 - 20%
Giúp tạo ra các hàm helper, tạo ra các hằng số để giúp việc vẽ trở nên thuận tiện	~ 30 - 35%
Sửa file CMakeLists.txt và hàm main() để sử dụng giao diện đồ họa của trò chơi	~ 30%
Hướng dẫn cách tương tác đồ họa với các keyboard/mouse event	~ 35 - 40%

2. Gemini:

Chủ yếu giúp đỡ về mặt cú pháp chương trình và giải thích ý nghĩa các hàm được sử dụng của chương trình. (Mức độ sử dụng: ~ 20%)

3. Github Copilot:

Sử dụng code autocompletion để hoàn thành những hàm lặp lại nhiều lần (drawText, drawRect, ...) và sử dụng comment từ đó gợi ý Copilot hoàn thiện code (ví dụ: // màu trắng thì Copilot sẽ tự biết là {255, 255, 255, 255}... từ đó tiết kiệm thời gian tra cứu và viết). (Mức độ sử dụng: ~20%)

VI. Các nguồn tham khảo khác:

Lazy Foo's SDL2 Tutorials: <https://lazyfoo.net/tutorials/SDL/>

SDL2 Wiki: <https://wiki.libsdl.org/SDL2/FrontPage>

SDL2_ttf Documentation: https://wiki.libsdl.org/SDL2_ttf/FrontPage

CMake Documentation: <https://cmake.org/cmake/help/latest/>

Source code và tài liệu hướng dẫn project-lv2.pdf - Viện Trí tuệ Nhân tạo, UET-ĐHQGHN (2026)